SYSTEM ARCHITECTURE | MILESTONES 1–3 COMPLETE | V0.1.0 SPECIFICATION

Multi-Agent Code Review System

Architectural Blueprint, Specialist Agent Contracts & Data Pipeline Specification

ARCHITECTURE PATTERN	PRIMARY CONTRACT	TEST COVERAGE	TARGET WORKFLOW
Multi-Agent Specialist	BaseAgent (ABC) & Finding	94% (124 Passing Tests)	GitHub PR & CLI Ingestion

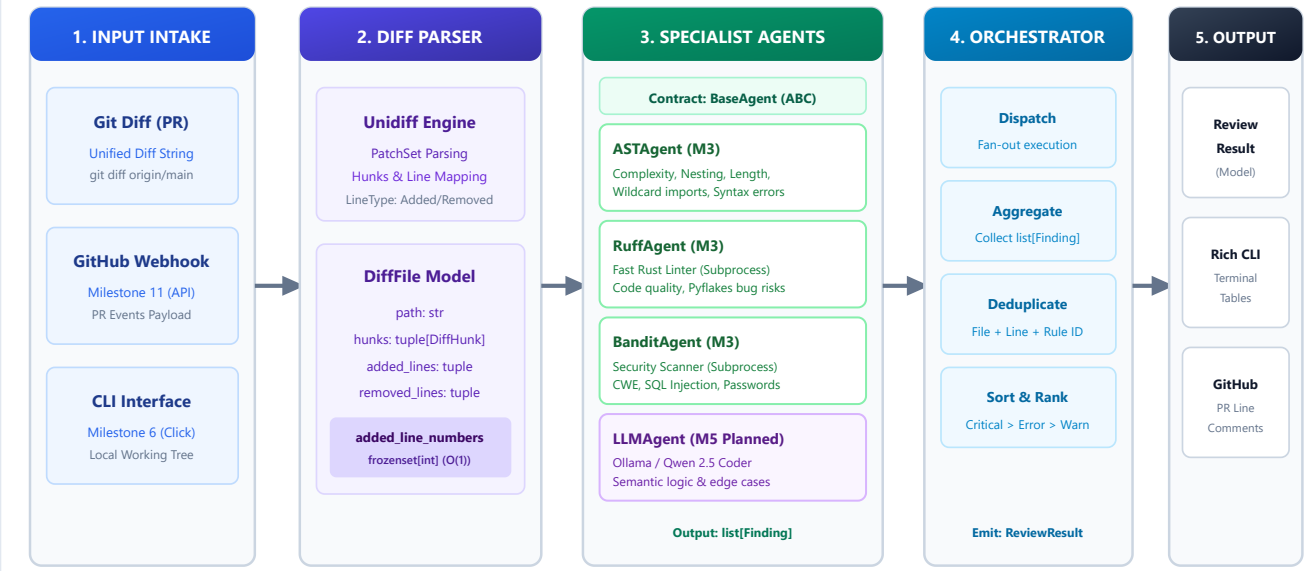
1. Executive Summary & Core Architectural Premise

Modern software development relies heavily on automated pull request (PR) reviews to ensure code quality, security, and maintainability. Monolithic AI reviewers attempt to evaluate entire codebases in a single, unstructured LLM prompt. This approach is prone to hallucinations, misses deterministic linting and security issues, incurs high API costs, and fails to provide precise line-level feedback.

The **Multi-Agent Code Review System** addresses this by emulating a real-world engineering team: reviewing code through a coordinated collective of **specialist agents**. Highly optimized, deterministic agents handle static syntax, style, and security vulnerabilities first, providing a zero-cost, reproducible baseline. Subsequent LLM agents (Milestone 5) focus exclusively on subtle logic flaws, API contract deviations, and semantic design concerns.

2. High-Level End-to-End System Architecture

FIGURE 1: LAYERED COMPONENT TOPOLOGY & INFORMATION FLOW — VECTOR SVG



3. Core Data Contracts & Communication Schemas

A decoupled multi-agent architecture requires strict, validated communication contracts. The review pipeline uses immutable Pydantic models (`frozen=True`) to eliminate side-effects and guarantee thread-safe concurrent execution across agents.

Finding (The Universal Currency)

`models/finding.py`

Represents one concrete issue. Every agent outputs `list[Finding]`, completely decoupling agents from the orchestrator and downstream report formatters.

Field	Type	Description
`file`	`str`	Repo-relative file path (forward slashes).
`line`	`int (ge=1)`	1-indexed line number in target file.
`severity`	`Severity`	`CRITICAL`, `ERROR`, `WARNING`, `INFO`.
`category`	`Category`	`SECURITY`, `BUG_RISK`, `CODE_QUALITY`, etc.
`agent`	`AgentType`	Agent origin: `AST`, `RUFF`, `BANDIT`, `LLM`.
`message`	`str`	Concise explanation of the defect.
`suggestion`	`str | None`	Actionable remediation recommendation.
`rule_id`	`str | None`	Identifier (e.g. `AST001`, `F401`, `B608`).

ReviewResult (Top-Level Container)

`models/review.py`

Aggregates findings across all files and agents, calculating summary metrics and providing convenience sorting helpers.

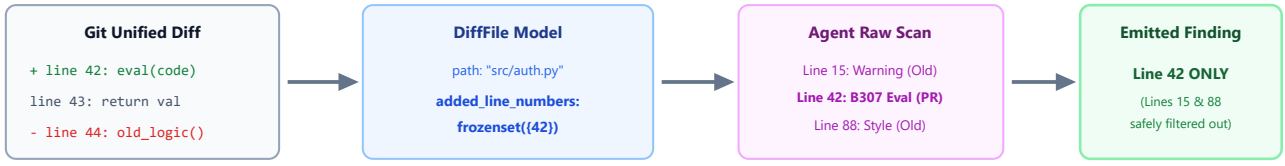
Attribute / Property	Type	Description
`findings`	`tuple[Finding]`	Immutable tuple of all findings.
`files_reviewed`	`int`	Total count of evaluated files.
`duration_seconds`	`float`	Total review pipeline wall-clock time.
`total_findings`	`int`	Computed total finding count.
`has_critical`	`bool`	True if any finding has `CRITICAL` severity.
`findings_by_severity`	`dict`	Counts mapped by `Severity` key.
`sorted_findings()`	`list`	Sorted by severity rank (descending).

4. Precision Line-Mapping & Diff Filtering Mechanics

> **Rule of Pull Request Review Hygiene:**
> Agents must never penalize authors for pre-existing legacy code that was not modified in the PR.

The `DiffParser` decomposes unified diff strings into `DiffFile`, `DiffHunk`, and `DiffLine` models. Each `DiffFile` calculates a cached `added_line_numbers: frozenset[int]` representing every newly added or modified line in the target file.

FIGURE 2: PRECISION LINE-FILTERING PIPELINE — VECTOR SVG



5. Deterministic Agent Deep Dive (Milestone 3)

ASTAgent — BUILT-IN AST

Technique: `ast.NodeVisitor`

Checks:

- `AST001`: Cyclomatic complexity > 10
- `AST002`: Nesting depth > 4
- `AST003`: Function length > 50 lines
- `AST004`: Wildcard `import *`
- `AST000`: Syntax errors

RuffAgent — RUST SUBPROCESS

Technique: `ruff check --output-format json`

Capabilities:

- Fast code style & formatting checks
- Pyflakes bug risks (undefined variables)
- Parses automated fix descriptions into actionable `suggestion` fields

BanditAgent — SECURITY LINTER

Technique: `bandit -f json -q`

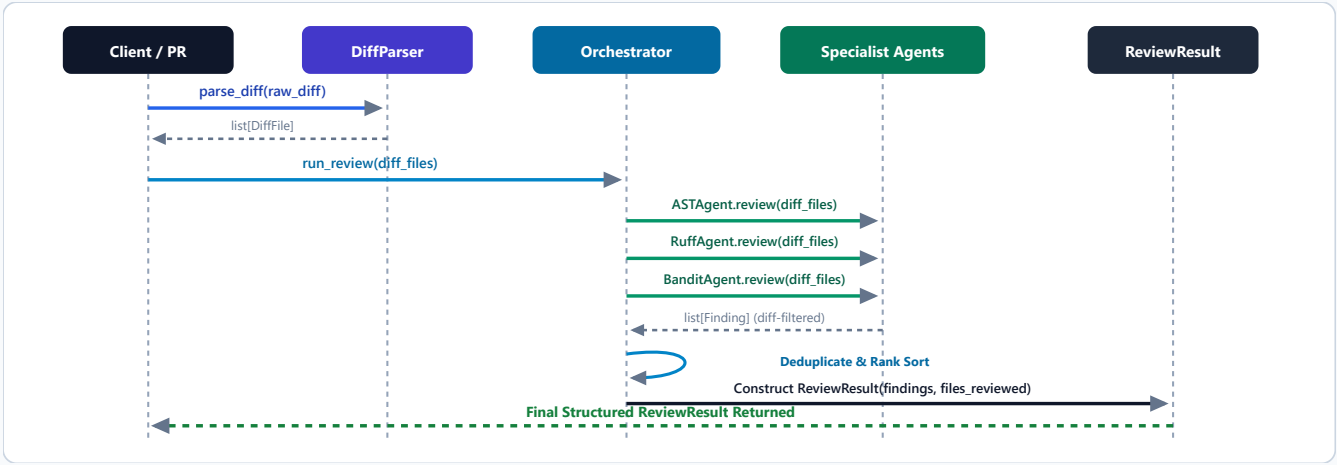
Capabilities:

- Hardcoded credentials (B105/B106)
- Command injection via `shell=True`
- Unsafe execution via `eval` (B307)
- SQL injection vector detection (B608)

6. Execution Lifecycle & Orchestrator Sequence (Milestone 4 Preview)

The sequence diagram below models runtime coordination across components. In Milestone 4, `ReviewOrchestrator` dispatches parsed diff files to all registered agents, collects findings, eliminates duplicate reports across tools, and generates the final structured `ReviewResult`.

FIGURE 3: RUNTIME EXECUTION SEQUENCE DIAGRAM — VECTOR SVG



7. Complete Implementation Roadmap

Milestone	Focus Area	Key Components	Status
M1	Project Foundation	`config.py`, `logging_setup.py`, pyproject.toml	COMPLETE
M2	Models & Diff Parser	`enums.py`, `finding.py`, `diff_parser.py`, `base.py`	COMPLETE
M3	Deterministic Agents	`ast_agent.py`, `ruff_agent.py`, `bandit_agent.py`	COMPLETE
M4	Orchestrator	`orchestrator.py`, finding deduplication, rank sorting	NEXT
M5	LLM Integration	Ollama + Qwen 2.5 Coder for semantic & logic review	Planned
M6	CLI Interface	Click CLI runner, Rich formatted terminal tables	Planned
M7	Reporting & Output	Markdown summary, JSON export, HTML report generator	Planned
M8–10	Perf, Cache & CI	Async concurrency, caching layer, GitHub Actions CI	Planned
M11	GitHub Integration	PR webhook receiver & automated line commenting	Planned